arXiv:2611.14949v1 [cs.AI] 11 Nov 2026

RSI-PDEbench: A Benchmark for Self-Improving Agents in PDE Solving

Binghang Lu[1,*], Lang Cao[2,†], Changhong Mou[3,†], Yige Sun[4], Cindy Xiangrui Kong[1], Zheyuan Deng[5], Tian Yuan[4], Bing Hu[4], Runyu Zhang[6], Na Lu[7] and Guang Lin[1,*]

[1]Purdue University, [2]University of Illinois Urbana-Champaign, [3]Utah State University, [4]Independent researcher, [5]Brown University, [6]Massachusetts Institute of Technology, [7]Arizona State University

Identifying and solving partial differential equations (PDEs) from sampled solution data often requires substantial expert effort in equation discovery, coefficient estimation, and numerical solver configuration, especially under noise and distribution shifts. Can large language model (LLM) agents automate this workflow accurately and efficiently? To investigate this question, we introduce RSI-PDEbench, a benchmark of 1,001 cases spanning 42 equation templates, including both classical and fractional PDEs. It provides three evaluation settings: Full evaluates performance on noise-free solution data; Noisy tests robustness to Gaussian noise with an RMS equal to 50% of the clean-signal RMS; and Hard-OOD tests challenging out-of-distribution generalization to unseen equation templates and shifted coefficient ranges. We further introduce RSI-PDE, a recursively self-improving agent harness that shortlists candidate equations by residual loss, uses relative L^2 feedback to refine coefficients and numerical solvers, and applies a validation gate to policy and memory updates. We compare RSI-PDE with single-run baselines and native agent harnesses (Claude Code or Codex) across four frontier models. Across the three settings, RSI-PDE improves the pass rate averaged across the four models by 106.5–181.9% relative to single-run baselines and by 3.0–19.6% relative to native harnesses, while reducing total recorded model calls by 46.9% across the 12 model–setting pairs. These results highlight the potential of validated recursive self-improvement in LLM agents to advance scientific computing.

github.com/jeremylu916/RSI-PDE jeremylu916.github.io/RSI-PDE JeremyLu1017/PDE-RSI

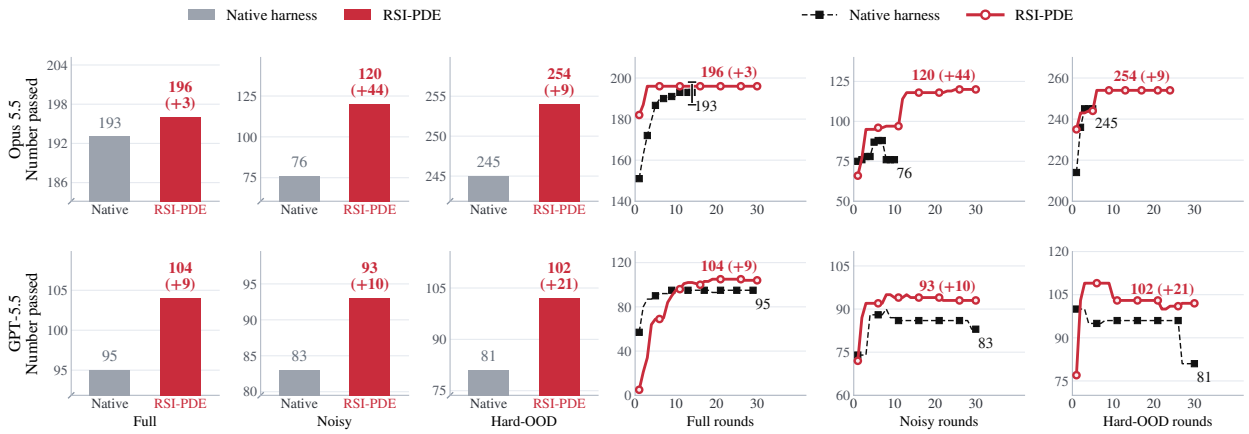

Figure 1 | **Final accuracy and learning curves.** Left: passed-case bars (gray: native; red: RSI-PDE). Right: learning curves (black: native; red: RSI-PDE). Both halves have Opus 5.5 above GPT-5.5, with Full, Noisy and Hard-OOD columns. Labels give final counts and RSI-minus-native case gains. Axes are locally zoomed; cropped bar baselines are marked. The native Opus Full curve whisker shows the observed 187–198 range over three runs. Curves preserve regressions and stopping rounds. Tables 3 and 4 give all models and baselines.

[*]*Corresponding authors: lu895@purdue.edu.* [†]*Equal contribution.*

1. Introduction

Scientific observations rarely arrive with a fully specified governing equation. Sparse field measurements and known forcing or boundary conditions may leave both the physical law and its coefficients uncertain. Noise and fractional dynamics make this identification harder. An accurate discretization of the wrong equation is insufficient, and a correct family with inaccurate coefficients can still produce a poor solution.

Coding agents can explore these possibilities through numerical experiments, but executable code alone is not evidence of scientific accuracy. Existing datasets support learning PDE solution operators and discovering equations from data (Takamoto et al., 2022; Gupta & Brandstetter, 2022; Hao et al., 2024; Ohana et al., 2024; Rudy et al., 2017). CodePDE studies solver synthesis for specified equations (Li et al., 2026a), and PDEAgent-Bench provides 645 specified-PDE solver-generation instances (Hang et al., 2026). We ask whether an agent can instead identify an unknown PDE or fractional PDE (FPDE), estimate its parameters and construct an accurate solver within a limited development budget.

We introduce RSI-PDEbench, a collection of **1,001 released PDE and FPDE cases across 42 templates** for this identify-then-solve task. To our knowledge, it is the largest joint PDE/FPDE benchmark for coding agents tasked with equation-family identification, parameter estimation and numerical solver construction. This scope concerns agent problem instances; it does not compare the number of simulated trajectories in general-purpose PDE learning datasets. The agent receives a finite equation library, 256 spatial–temporal samples and known forcing, initial and boundary functions, while the true family and coefficients remain hidden for a new case. This is a controlled analogue of scientific system identification, with classical, nonlinear, multiphysics and fractional dynamics in a common executable interface.

Three complementary settings expose distinct sources of uncertainty. **Full** tests all 42 templates with noise-free observations. **Noisy** adds Gaussian perturbations at 50% of the clean-signal RMS. **Hard-OOD** combines unseen templates and coefficients shifted beyond development ranges. Together, they probe instance generalization, observation robustness and transfer to unfamiliar physics or parameters. Their held-out counts are distinct from the 1,001-case release.

The **RSI-PDE harness** organizes exploration through residual guidance and reference-based validation. Inside a sandbox, the agent substitutes a reconstructed or computed field into candidate equations, fits coefficients and retains up to five low-residual families. Forward solves and public relative L^2 feedback then distinguish hypotheses and refine coefficients and discretizations. A gate retains improvements to the solver, its instruction policy and memory, while the foundation model and executable supervisor stay fixed. Residuals narrow the family search; relative L^2 tests assess the resulting solution. Hidden references never guide online equation choice.

We compare single-run programs, native harnesses (Claude Code or Codex), and RSI-PDE using Opus 5.5, GPT-5.5, GPT-6.1 and Sonnet 5.5. Across the four models, RSI-PDE increases mean pass rate over single-run baselines by 181.9%, 140.2% and 106.5% on Full, Noisy and Hard-OOD, and over native harnesses by 3.0%, 19.6% and 5.4%, respectively. Recorded model calls sum to 1,884 rather than 3,547 across the 12 comparisons, a 46.9% reduction, although calls and dollar costs are not lower in every comparison. Figure 1 shows the final counts and learning curves for Opus 5.5 and GPT-5.5. These are descriptive results from the reported runs; replication, adaptive benchmark reuse and cost provenance are discussed in Section 6.

Our contributions are:

- **A large joint PDE/FPDE benchmark:** 1,001 released cases spanning 42 equation templates, with analytic or high-accuracy numerical references and quantitative solution checks.
- **Three diagnostic evaluation settings:** Full, Noisy and Hard-OOD separately test noise-free accuracy, robustness to 50% Gaussian sample noise, and unseen templates with shifted coefficients.
- **An unknown-family, unknown-parameter task:** agents must explore a finite physical hypothesis space, estimate coefficients and return an executable solver from sparse observations and known inputs.
- **A residual-guided improvement harness:** equation residuals prioritize a five-candidate shortlist, while public relative L^2 feedback and validation retain improvements to equation selection, coefficient estimation, numerical solution, policy and memory.

The remainder introduces the task and evaluation interface (Section 2), dataset construction (Section 3), the harness (Section 4) and experiments (Section 5). Related work and implementation details appear in the appendix. Code, data and the result website are linked on the first page.

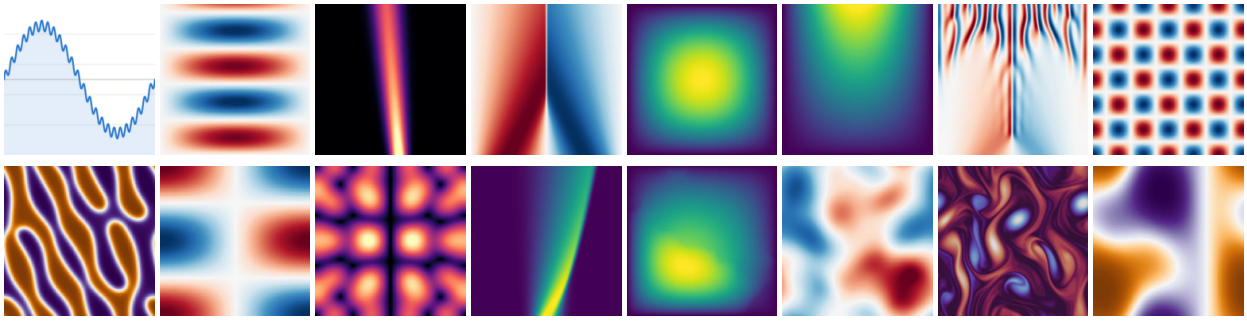

Figure 2 | Examples of reference fields from public training cases. The benchmark includes diffusion and transport, fractional dynamics, wave propagation, fluid flow, phase-field systems and electromagnetics. The gallery is illustrative; Section B gives the complete 42-template catalog.

2. Benchmark Overview

2.1. Task scope: unknown equation and unknown coefficients

A case is generated from

$$\mathcal{N}_{k,\theta}[u](x,t) = f(x,t), \qquad (x,t) \in \Omega \times (0,T], \tag{1}$$

with the corresponding steady or ordinary-differential specialization. The hidden index k selects a PDE/FPDE from the supplied class library $\mathcal{K}$, and θ denotes its unknown physical coefficients, including fractional-order parameters where applicable. The agent must infer both before choosing and implementing a suitable numerical method. Template names in the library define the allowed hypothesis space; they do not reveal the class of an individual hidden case.

This formulation captures a difficult part of practical scientific modeling: observations can fit several candidate laws, and model mismatch, parameter error and discretization error can look similar. It remains a controlled finite-library task, rather than unrestricted discovery of an arbitrary physical law. The benchmark makes these uncertainties explicit so that exploration and numerical repair can be evaluated together.

2.2. Agent-facing inputs and executable interface

The public case view contains the domain, known callable forcing, initial and boundary functions, evaluation-grid specifications, the equation library, and 256 spatial–temporal samples,

$$\mathcal{D}_u = \{(z_i, y_i)\}_{i=1}^{256}, \qquad y_i = u^\star(z_i) + \epsilon_i. \tag{2}$$

Here $z_i = (x_i, t_i)$ denotes a sample location, specialized to spatial-only or time-only coordinates where appropriate. Full sets $\epsilon_i = 0$; the other settings perturb observed values as described in Section 3.2.

An entrant writes `solve(problem)`. Its program identifies a candidate class and coefficients and returns a numerical solution on its chosen grid, or at prescribed points for the Elliptic equation problem. The evaluator maps the returned field to its evaluation grid. Class and coefficient estimates are diagnostic outputs; a correct class name without an accurate field does not pass. The entrant can choose classical, learned or hybrid numerical methods within the sandbox and resource contract.

Training diagnostics can reveal class and coefficient labels. Validation returns numerical diagnostics but not the hidden class or coefficients. Hidden test labels, reference arrays and scores are inaccessible during agent execution. A retained program must therefore identify each new test case from its public inputs, without selecting an equation using that case's hidden reference error.

2.3. Accuracy, executability and reference solutions

The reference field $u^\star$ defines the numerical scoring target. For analytic cases, the evaluator computes its values directly at evaluation nodes. Other references are produced with refined meshes, suitable higher-order discretizations or time integration, and strict solver tolerances. Fine/coarse calculations mapped to a common grid check self-convergence. Reference generation and thresholds are fixed independently of an entrant's solver; Section C gives representative settings and their limitations.

Table 1 | Declared budget per development sub-run. All aggregate resource figures sum the three sub-runs before averaging repeated runs. Final hidden evaluation is outside the development budget.

Resource	Cap
Development wall-clock	2 h
Model requests	400
Prompt tokens, including cache reads/writes	40 M
Output tokens	1.5 M
Validation submissions	30
Solver resources	1 CPU core + 1 H100 GPU

Let $\mathcal{I}\hat{u}$ be the submitted solution interpolated to the reference grid. For time-dependent problems,

$$E_{\mathrm{st}} = \frac{\|\mathcal{I}\hat{u} - u^\star\|_{L^2(\Omega\times[0,T])}}{\|u^\star\|_{L^2(\Omega\times[0,T])}}, \qquad E_T = \frac{\|\mathcal{I}\hat{u}(\cdot,T) - u^\star(\cdot,T)\|_{L^2(\Omega)}}{\|u^\star(\cdot,T)\|_{L^2(\Omega)}}. \tag{3}$$

The norms use the evaluator's quadrature weights; steady and point-cloud cases use their corresponding norms. The submitted program passes only if both applicable errors are at most $\tau = 2 \times 10^{-3}$, initial and boundary conditions satisfy the prescribed 10^{-10} check, the returned arrays meet the contract, and the solve meets runtime and memory limits. The Elliptic equation problem has a separately fixed 5×10^{-2} tolerance because its reference discretization is less accurate. The model does not receive the hidden reference arrays. Public training and validation errors are computed by the supervisor.

Pass count and pass rate summarize end-to-end success. Relative L^2 diagnostics distinguish numerical accuracy from execution or contract failure, while model calls, tokens, validation submissions and dollar cost characterize development effort. Class labels and parameter estimates help diagnose failures but are not substitutes for the solution-error criterion.

2.4. Budgeted development and frozen testing

A complete experiment combines three development sub-runs organized by solver-library group. Each receives the allowance in Table 1; groups may run in parallel. Across three groups the declared ceilings are 1,200 model calls, 120 M prompt tokens, 4.5 M output tokens and 90 validation submissions. The two-hour allowance applies to each group. Aggregate usage can therefore exceed a single group's 400-call limit.

A proxy records model identity and usage. Model-written code runs in a separate sandbox with bounded imports, filesystem access, network access, runtime and memory. Agents can request training experiments within the development budget; a validation submission is counted when evaluation starts. The benchmark-native wrapper retains accepted programs using the reference-error gate. Section 4.4 describes the RSI-PDE retention mechanism, including its public-pass rule.

After development, an independent evaluator scores the frozen retained program on hidden cases. For a combined learning curve at round r, each group contributes its retained program after r validation submissions, or its final program if it stopped earlier. Thus a round is a submission index, not a common elapsed-time budget, and summed group submissions differ from the final index of a combined curve. Hidden checkpoint scores are assembled for reporting and do not return to the running agent.

3. Dataset Construction

3.1. Case inventory and equation coverage

RSI-PDEbench releases 1,001 cases and covers 42 templates. A template specifies an equation and a construction of coefficients, forcing, initial/boundary data and reference solution; an individual case instantiates that construction. Multiple templates may share one equation family, so 42 templates does not mean 42 unrelated physical laws. The catalog contains 20 classical and fractional templates, 10 nonlinear or multiphysics templates, and 12 data-driven or additional-dimensional templates (Section B).

Classical coverage includes Poisson, heat, convection–diffusion, Burgers and wave equations. Fractional coverage includes a Caputo ODE, time-fractional diffusion, multi-term fractional dynamics and orders varying in time or space–time. These problems test whether identification routines can distinguish local dynamics from

Table 2 | The three current evaluation settings. Noisy contains 200 selected cases in total, of which 160 are test cases. Hard-OOD separates 188 cases from unseen templates and 90 cases with shifted coefficients.

Setting	Templates	Train	Validation	Test	Observation noise
Full	42	42	42	210	None
Noisy	24	22	18	160	50% RMS
Hard-OOD	39	20	20	278	1% RMS

memory-dependent operators and estimate their coefficients or orders. Nonlinear and multiphysics coverage includes Kuramoto–Sivashinsky, Navier–Stokes, Cahn–Hilliard, Maxwell and time-harmonic electromagnetics. Additional templates include Darcy, Helmholtz, Allen–Cahn, two-dimensional Burgers and advection–diffusion–reaction equations.

Spatial dimensions are one or two, with the fractional ODE as the zero-spatial-dimensional specialization. The collection mixes manufactured solutions, freely evolving initial conditions, sharp layers, multiscale fields and nonlinear dynamics. Five templates reuse input fields from DiffusionPDE (Huang et al., 2024); their reference solutions are recomputed for this benchmark. These fields add variation in coefficients or inputs without making the full collection a dataset of experimental measurements. Figure 2 illustrates public training fields.

3.2. Full, Noisy and Hard-OOD settings

The three settings probe different sources of uncertainty. They are not an exhaustive partition of the 1,001 released cases, and their pass rates describe different case distributions. Full draws a balanced noise-free evaluation across the catalog, Noisy selects a diagnostic portion of the public release, and Hard-OOD uses a separate sealed draw. Their training, validation and hidden-test allocations are given in Table 2.

Full: noise-free instance generalization. Full uses the `full0` sampler, with no observation noise. Each of the 42 templates contributes one training, one validation and five hidden-test cases: 42/42/210 in total. Public seeds determine training and validation instances, while a separate secret determines hidden tests. It measures generalization to new instances from represented template distributions.

Noisy: severe observation corruption. Noisy selects 200 cases found easiest by fixed calibration probes and adds Gaussian perturbations with standard deviation equal to 50% of the clean-signal RMS. The selected cases cover 24 templates and yield 22 training, 18 validation and 160 test instances. A template with at least three selected cases contributes to all splits; two-case templates contribute training and test instances, and one-case templates contribute test instances. Selection helps distinguish observation robustness from baseline numerical difficulty. Because these cases come from a regenerable public release, this setting is a diagnostic test rather than a fresh secret holdout.

Hard-OOD: unseen templates and shifted coefficients. A sealed draw over 39 templates provides 20 training and 20 validation cases from 20 seen templates. Its 278 hidden cases combine 188 instances from 19 templates absent from both development splits and 90 instances with coefficients in the opposite half of the range used in development. The latter come from the seen templates with an identifiable coefficient; the two seen templates without such a coefficient contribute no shifted cases. Observation noise is 1% RMS and its level is hidden. The equation library is still supplied, so unseen means absent from training and validation, not absent from the allowed library. The task text discloses both forms of shift. A problem-specific feasibility filter conditions the draw, limiting the population to which its results generalize.

3.3. Reference validation and reproducibility

Case generation keeps stable identifiers, template lineage and separate public/evaluator views. Analytic references are sampled directly on the scoring grid; numerical references use family-appropriate refinement and convergence checks (Section C). Fixed solver probes inform calibration and hardening, but do not change the acceptance tolerance for an individual entrant. The same frozen evaluator checks all reported programs for a setting.

The release, split construction, numerical references and frozen result snapshot support separate inspection of task generation, solver behavior and reported aggregates. Neither the 1,001-case inventory nor the held-out

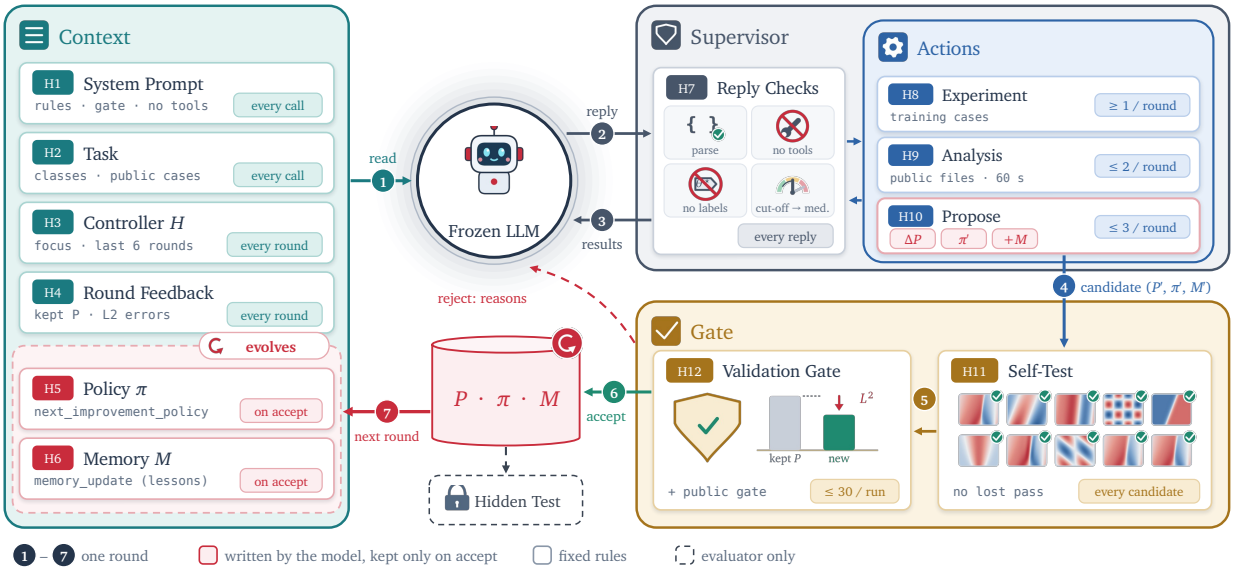

Figure 3 | **RSI-PDE harness.** The frozen LLM reads the context (H1–H6), and its reply passes the supervisor checks (H7) before becoming a training experiment (H8), a public-data analysis (H9) or a proposal (H10). A proposal updates solver P, instruction policy π and memory M. The candidate passes the training self-test (H11) and the validation or public-pass gate (H12); the text describes the bounded training-regression waiver. Accepted state returns in the next round's context; rejected proposals return diagnostic feedback. The supervisor and model weights remain fixed. Only the independent evaluator scores the frozen solver on hidden tests.

pass count measures unrestricted scientific coverage: dimensions, templates, admissible parameters and feasibility conditioning define the benchmark's scope.

4. RSI-PDE Harness

4.1. Fixed supervisor and evolving solver, policy and memory

The RSI-PDE harness structures an agent's numerical exploration while keeping its foundation model and executable supervisor fixed. Its accepted state is (P, π, M): P is a modular solver with an identification router, equation-class modules and shared helpers; π is a model-written instruction policy for subsequent investigation; and M stores reusable numerical lessons. Figure 3 shows how that state moves through one round. Figure 4 separately follows the physical identification and refinement loop.

Each round supplies the active program, policy, memory, recent history, public diagnostics and remaining resources. The agent can analyze public files, experiment on training cases and propose solver/policy/memory updates. The supervisor requires a training experiment before a proposal and bounds the round to at most two analyses and three proposal attempts. Reply parsing, sandbox execution, label-copy checks, no-op filtering and training self-tests constrain these actions before a validation submission is spent. Numerical analysis runs inside the solver sandbox; it does not grant the model unrestricted host execution.

4.2. Residual guidance for PDE and FPDE family search

The agent begins with an available field u_a, obtained from its current solver or a regularized reconstruction of public observations. It substitutes this field into candidate equations inside the sandbox and fits their unknown coefficients. For candidate class k, write the equation as signed terms, including the forcing:

$$\mathcal{R}_{k,\theta}[u_a] = \sum_j T_{k,j}[u_a;\theta], \qquad r_{\mathrm{id}}(k,\theta) = \frac{\|\mathcal{R}_{k,\theta}[u_a]\|_Q}{\sum_j \|T_{k,j}[u_a;\theta]\|_Q + \varepsilon}, \quad \mathcal{L}_{\mathrm{res}} = r_{\mathrm{id}}^2. \tag{4}$$

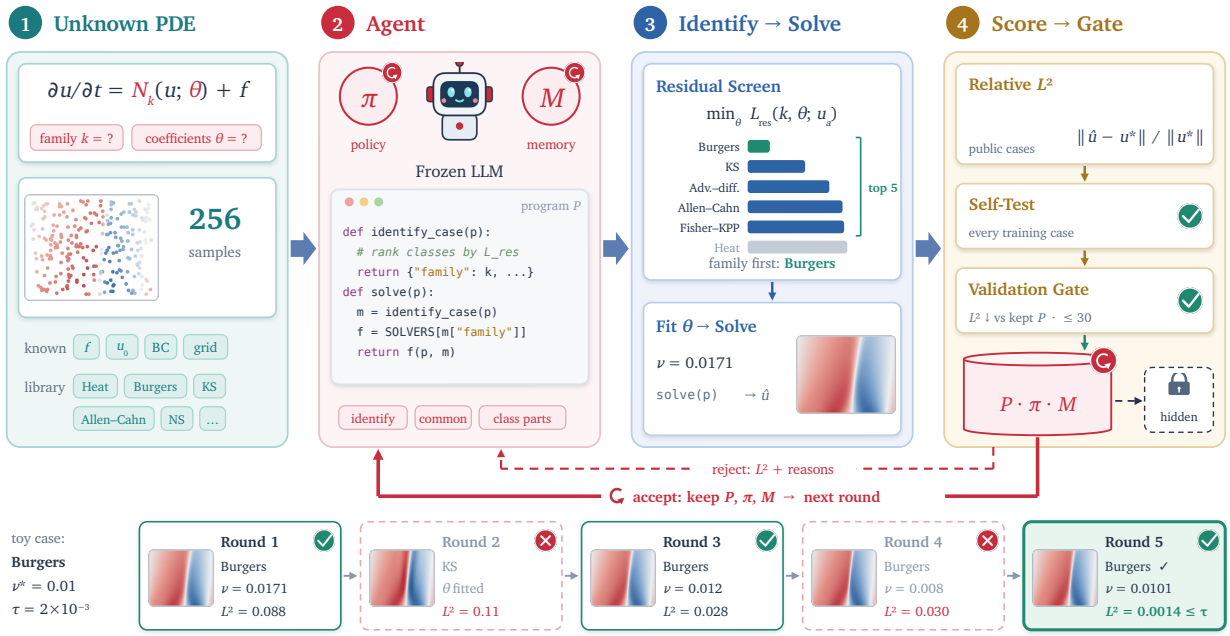

Figure 4 | **The residual-guided RSI-PDE workflow.** (1) The agent starts without the equation: the family k and the coefficients θ are unknown, and it receives 256 samples, the known forcing, initial and boundary data, and the class library. (2) A frozen LLM, steered by its self-written policy π and memory M, writes the program P, whose router identifies the case and dispatches it to a class solver. (3) In the sandbox, residual ranking by $\min_\theta \mathcal{L}_{\text{res}}(k, \theta; u_a)$ ranks candidate families and keeps the five best; forward solves and public relative L^2 checks then refine the equation choice and its coefficients. (4) Public relative L^2 errors, the self-test and the validation gate decide what is kept. Accepted (P, π, M) carry into the next round, so the policy and memory evolve with the solver; a rejection returns its L^2 errors and reasons. The bottom row follows a toy viscous Burgers case ($\nu^\star = 0.01$): the residual fit on a smoothed field gives $\nu = 0.0171$, and gated L^2 feedback refines it to $\nu = 0.0101$, below τ. The later coefficient values are illustrative; the residuals, fields and errors are computed for this toy case, not taken from benchmark runs.

Here Q denotes the usable collocation grid or weak-test set, and ε protects against a zero normalizer. Coefficients are fitted within the candidate's admissible ranges. Class-specific objectives may also include sample-fit and boundary terms. The candidate law and public inputs define this calculation; neither the hidden true equation nor a hidden reference field is required.

Residual ranking prioritizes at most five hypotheses:

$$C_5 = \text{TopK}_{k \in \mathcal{K}} \left[-\min_\theta \mathcal{L}_{\text{res}}(k, \theta) \right], \qquad K = \min(5, |\mathcal{K}|). \tag{5}$$

This screening focuses subsequent forward-solver experiments on a small set of plausible PDE/FPDE families. It avoids committing to one equation solely because it fits sparse observations. For noisy data, numerical differentiation can amplify perturbations; the agent can use smoothed reconstructions, solver-generated fields, spectral representations or weak residuals suited to the candidate operator. Term normalization and known initial/boundary data help distinguish physical consistency from an artificially favorable term scale.

The purpose of residual guidance is to reduce unstructured family search. The aggregate experiments do not isolate its wall-clock benefit or top-five recall, and a low residual does not guarantee that the correct law survives the shortlist. Residual computation remains an agent-written numerical procedure, rather than a supervisor oracle revealing the true equation.

4.3. Relative L^2 feedback and coefficient refinement

For each shortlisted family, the agent implements or revises a forward solver, estimates coefficients and checks its predicted field. Residual and sample-fit evidence suggest candidate parameters; public training relative L^2

Algorithm 1 Residual-guided solver development in RSI-PDE

Input: fixed model, public library and data, development budget
Initialize solver P, instruction policy π and memory M
while development resources and validation submissions remain **do**
  Obtain public diagnostics and the current field approximation u_a
  In the sandbox, fit candidate equations and evaluate $\mathcal{L}_{\text{res}}$
  Form or refresh the shortlist C_5 of at most five equations
  Allocate experiments to shortlisted solvers; compare residuals and training L^2 errors
  Refine the equation, coefficients and numerical method; propose (P', π', M')
  Apply static checks, training self-tests and proposal filters
  if the candidate is eligible for validation **then**
    Evaluate reference-based validation errors and apply the acceptance gate
    if accepted **then**
      Merge the accepted program edits and corresponding policy/memory updates
    end if
  end if
  Record diagnostics, proposal lineage and resource use
end while
Freeze the retained solver; evaluate test accuracy and post-hoc physical residuals

errors show whether the resulting solution is accurate. Repeated experiments can distinguish a wrong family from a poorly fitted coefficient or an under-resolved numerical method. Known forcing, initial and boundary functions provide additional constraints on an otherwise ambiguous inverse problem.

The combination of residual and relative L^2 information is staged: residual loss prioritizes scientific hypotheses, then reference-based tests guide equation choice, coefficient estimation and discretization repair on public cases. The validation gate evaluates whether a proposed reusable program improves. There is no arbitrary weighted residual-plus-L^2 threshold in the pass criterion. The desired outcome is a coefficient estimator and solver that recover accurate solutions on new instances; the solution pass rate alone does not establish exact coefficient recovery when different parameters are observationally similar.

At hidden inference time, the frozen router must choose from public inputs, residuals and sample-fit evidence. It cannot request the test reference or its relative L^2 error. Public development errors improve the identification procedure; they do not provide an online oracle for selecting the hidden test equation.

4.4. Validation, retention and feedback

The standard reference-error gate requires a greater than 0.5% decrease in macro relative L^2 error, no class-average increase above 5%, no loss of class passes, no per-case space–time or final-time error increase above 25%, and valid execution and initial/boundary data. Class-local changes can be gated and merged independently; shared router/helper changes are evaluated at program level.

The harness also uses a public-pass rule over training and validation cases when the standard gate rejects a candidate. That rule requires at least one gained pass, no lost pass, no newly broken execution or constraint/resource failure, and at most a 25% error increase for cases passing in both programs, subject to the implementation's numerical floor. The all-training self-test normally preserves existing training passes. After two consecutive rounds blocked by that guard, it can waive the training-regression rule for one round; the validation/public-pass decision still applies. Thus an update can address a remaining public failure without being retained solely for reducing errors on already accurate cases.

Accepted solver edits carry their corresponding policy and memory updates into the next round. Rejected candidates return their numerical errors and rejection reasons, while remaining in the audit history. Algorithm 1 summarizes this process. The solver, policy and memory evolve; the displayed experiments do not promote self-modified executable supervisors. Establishing improvement of the controller itself would require equal-budget paired continuations from the same starting solver and memory.

5. Experiments

5.1. Experimental setup and baselines

We compare four foundation models—Opus 5.5, GPT-5.5, GPT-6.1 and Sonnet 5.5—under three modes:

- **Single run:** one model response per development group, frozen without iterative repair.
- **Native harness:** Claude Code for the Claude models and Codex for the GPT models, with execution and repair under the benchmark wrapper and declared development limits.
- **RSI-PDE harness:** structured numerical experiments and gated solver, policy and memory updates as described in Section 4.

Model weights stay fixed within each model-specific comparison. Native and RSI-PDE modes share the declared resource ceilings and independent numerical evaluator, but their tools and prompting differ; this is a system comparison rather than an ablation of one component. We report pass counts, relative L^2 diagnostics, learning curves and resource use separately.

The numerical record is the frozen public website snapshot dated November 7, 2026, including corrected GPT-6.1 Noisy identities. Most harness cells contain one run; native Opus 5.5 on Full averages three runs, with 187–198 passes. Single-run means use the available response samples for each model and setting. The Opus single-run baseline uses 128k output tokens. Where an answer was re-requested at the model's maximum output length, it replaces the answer truncated by a shorter limit; incomplete one-response programs score zero. Unequal replication and output allowances should be considered when interpreting the comparisons.

Author-reported single-run totals are GPT-6.1 Hard-OOD 135/278 and Sonnet 5.5 Full 59/210, Noisy 36/160 and Hard-OOD 104/278. Their missing validation/resource breakdowns are not inferred. Full native GPT-6.1 and Sonnet 5.5 headline totals are 189 and 183, with curves supplied as rounded percentages. Their earlier per-template, residual and resource measurements remain explicitly labeled diagnostic records, rather than being reconstructed to match these totals. GPT-6.1 Noisy uses the corrected attribution throughout: Codex 90/160 and RSI-PDE 97/160. The snapshot preserves raw record identifiers and source provenance; Section D describes reproduction.

Aggregate improvement. Across the four models, mean pass rates for single run/native/RSI-PDE are 28.7/78.6/81.0% on Full, 26.5/53.3/63.8% on Noisy and 37.2/72.9/76.9% on Hard-OOD. Relative gains are computed as $100(\bar{p}_{\mathrm{RSI}} - \bar{p}_{\mathrm{baseline}})/\bar{p}_{\mathrm{baseline}}$, using the mean of the four model pass rates within each setting. These yield 181.9/140.2/106.5% gains over single run and 3.0/19.6/5.4% over native harnesses. The corresponding absolute native-to-RSI-PDE gains are 2.4/10.5/4.0 percentage points. Relative gains and percentage-point gains therefore have different scales.

5.2. Full benchmark

Table 3 gives all four models. RSI-PDE reaches 196/210 (93.3%) with Opus 5.5, 104/210 (49.5%) with GPT-5.5, 193/210 (91.9%) with GPT-6.1 and 187/210 (89.0%) with Sonnet 5.5. The respective native counts are 193, 95, 189 and 183. The observed margins are therefore 3, 9, 4 and 4 cases, or approximately 1.4, 4.3, 1.9 and 1.9 percentage points. These are descriptive differences, not significance results; the three-case Opus difference is smaller than the native harness's observed 11-case range.

Across all four models, iterative harnesses add approximately 57–130 passes relative to the available single-run baselines. For Sonnet 5.5, the native harness improves from 59 to 183 passes and RSI-PDE to 187. This supports the practical value of execution and repair under numerical feedback. It does not isolate a particular residual, memory or acceptance rule: the system comparison changes multiple aspects of exploration and feedback together.

The curves in Figure 5 show strong early gains and later plateaus for several models. They are most useful as records of retained solver quality: repeated public submissions and unchanged programs can leave long flat segments. Since sub-runs can stop at different times, the horizontal axis does not establish that one system reached an accuracy level faster in seconds.

Table 3 | Full results. Passes are out of 210; gain over the single-run baseline and validation-minus-test gap are percentage points. Bold marks the higher displayed harness count for a model. †: author-updated native total with earlier-run resource/diagnostic records. ∗: retrospective stop-adjusted RSI cost, not the cost of the full-budget score shown.

Model	Entrant	Passed /210	Rate (%)	Runs	Gain (pp)	Gap (pp)	Calls	Cost ($)
Opus 5.5	Single run	66	31.5	4	–	–	3	7.32
	Claude Code	193	91.9	3	+60.4	+2.5	240	16.74
	RSI-PDE	**196**	93.3	1	+61.8	-0.5	88	30.47*
GPT-5.5	Single run	38	18.2	4	–	–	3	1.22
	Codex	95	45.2	1	+27.0	+7.1	482	40.97
	RSI-PDE	**104**	49.5	1	+31.3	+5.2	88	32.01
GPT-6.1	Single run	77	37.0	4	–	–	4	–
	Codex†	189	90.0	1	+53.0	+10.0	548	–
	RSI-PDE	**193**	91.9	1	+54.9	+5.7	146	–
Sonnet 5.5	Single run	59	28.1	1	–	–	–	–
	Claude Code†	183	87.1	1	+59.0	+3.3	148	6.41
	RSI-PDE	**187**	89.0	1	+61.0	+3.8	95	21.81*

Table 4 | Noisy and Hard-OOD results for all four models. Bold marks the higher harness count, including ties. Unseen and shifted counts partition Hard-OOD. ‡: corrected GPT-6.1 Noisy attribution; the mark applies to the Noisy identity and its corresponding calls. Missing single-run group breakdowns remain unavailable rather than inferred.

Model	Entrant	Noisy /160	Calls	Hard-OOD /278	Calls	Unseen /188	Shifted /90
Opus 5.5	Single run	37	3	133	3	88	45
	Claude Code	76	187	245	240	161	84
	RSI-PDE	**120**	278	**254**	81	**165**	**89**
GPT-5.5	Single run	51	3	42	3	33	9
	Codex	83	332	81	374	44	37
	RSI-PDE	**93**	104	**102**	91	**57**	**45**
GPT-6.1	Single run	45	3	135	3	88	47
	Codex‡	90	140	**261**	643	**175**	86
	RSI-PDE‡	**97**	539	**261**	111	173	**88**
Sonnet 5.5	Single run	36	3	104	3	60	44
	Claude Code	92	117	224	96	134	**90**
	RSI-PDE	**98**	138	**238**	125	**148**	**90**

5.3. Severe noise and out-of-distribution generalization

On Noisy, RSI-PDE passes 120/160 with Opus 5.5, 93/160 with GPT-5.5, 97/160 with GPT-6.1 and 98/160 with Sonnet 5.5, compared with 76, 83, 90 and 92 for the native harnesses. The largest observed margin is 44 cases with Opus 5.5. The corrected GPT-6.1 comparison favors RSI-PDE by seven cases. These differences suggest that investigation and coefficient estimation are particularly consequential under heavy observation noise, but a residual-shortlist ablation is needed to attribute that effect to the proposed identification procedure.

Hard-OOD yields 254/278 versus 245 with Opus 5.5, 102 versus 81 with GPT-5.5, 261 versus 261 with GPT-6.1, and 238 versus 224 with Sonnet 5.5. The reported GPT-6.1 single-run baseline passes 135/278, so both its native and RSI harnesses add 126 cases (45.3 percentage points), while remaining tied with each other. On the 90 shifted-coefficient cases, RSI-PDE passes 89, 45, 88 and 90, while the native harnesses pass 84, 37, 86 and 90. Thus it is no worse on the displayed shifted comparisons, with strict gains for three models and a tie for Sonnet 5.5. The unseen-template comparison is less uniform: GPT-6.1 favors Codex by two cases (175 versus 173), despite the equal aggregate result.

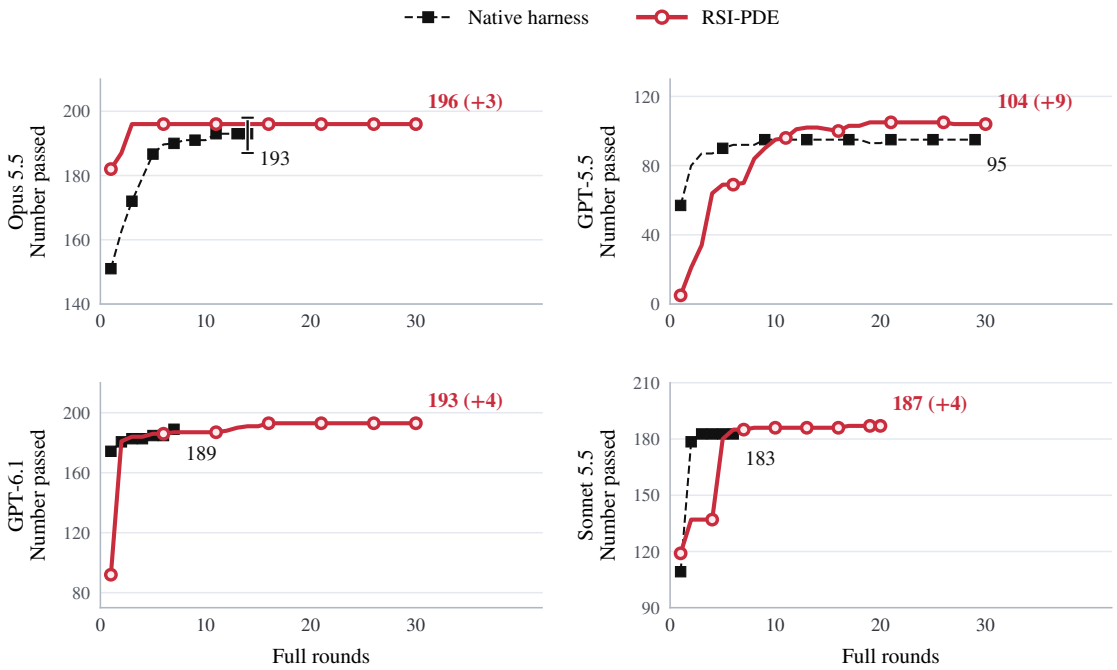

Figure 5 | Full passed-case trajectories for all four models. Black dashed squares: native harness; red circles: RSI-PDE. Local vertical ranges show recorded points from round one. Final labels give headline counts; red labels also give RSI-minus-native gains in cases. The native Opus 5.5 whisker shows the observed 187–198 range over three runs. GPT-6.1 and Sonnet 5.5 native count trajectories are approximate, converted from the author's rounded percentages; the Sonnet endpoint therefore need not equal 183 before rounding. A round index is not a common wall-clock budget.

These observations motivate keeping equation selection, coefficient identification and forward discretization as separate diagnostics. A transferable identification routine must fit a new instance's parameters, while a broad solver library must still route templates absent from development. Hard-OOD is conditioned by its construction and each cell has one run, so the current results do not establish universal transfer behavior.

5.4. Model calls and cost

Across the 12 model–setting pairs, recorded calls sum to 3,547 for native harnesses and 1,884 for RSI-PDE, a 46.9% reduction. On Full, the native-to-RSI call ratios are approximately 2.7, 5.5, 3.8 and 1.6 for Opus 5.5, GPT-5.5, GPT-6.1 and Sonnet 5.5. Across all 12 model-setting pairs, RSI-PDE uses fewer recorded calls in eight. The exceptions include three Noisy comparisons: 278 versus 187 calls with Opus 5.5, 539 versus 140 with GPT-6.1, and 138 versus 117 with Sonnet 5.5. On Hard-OOD, Sonnet 5.5 uses 125 versus 96 calls. A structured loop therefore changes the allocation of computation, rather than guaranteeing fewer calls.

Dollar cost depends on token lengths, caching and stopping behavior as well as call count. Full GPT-5.5 costs $32.01 with RSI-PDE versus $40.97 with Codex, a 21.9% reduction using recorded costs. That measured comparison does not imply lower cost for every model or setting. The website substitutes a *retrospective stop-rule replay cost* for selected RSI rows: it prices the calls before the replayed saturation stop, while keeping the full-budget pass rates, calls and trajectories. These costs, marked $*$, are useful estimates of spending that a rule might save, but are not jointly measured accuracy/cost outcomes from a fresh stopped run. For example, Full Opus 5.5 is listed at $30.47 under that replay, whereas its 196 passes come from the complete selected run. GPT-6.1 has no dollar estimate in the current public snapshot. We preserve unavailable values and avoid deriving a universal cost-efficiency claim from mixed provenance.

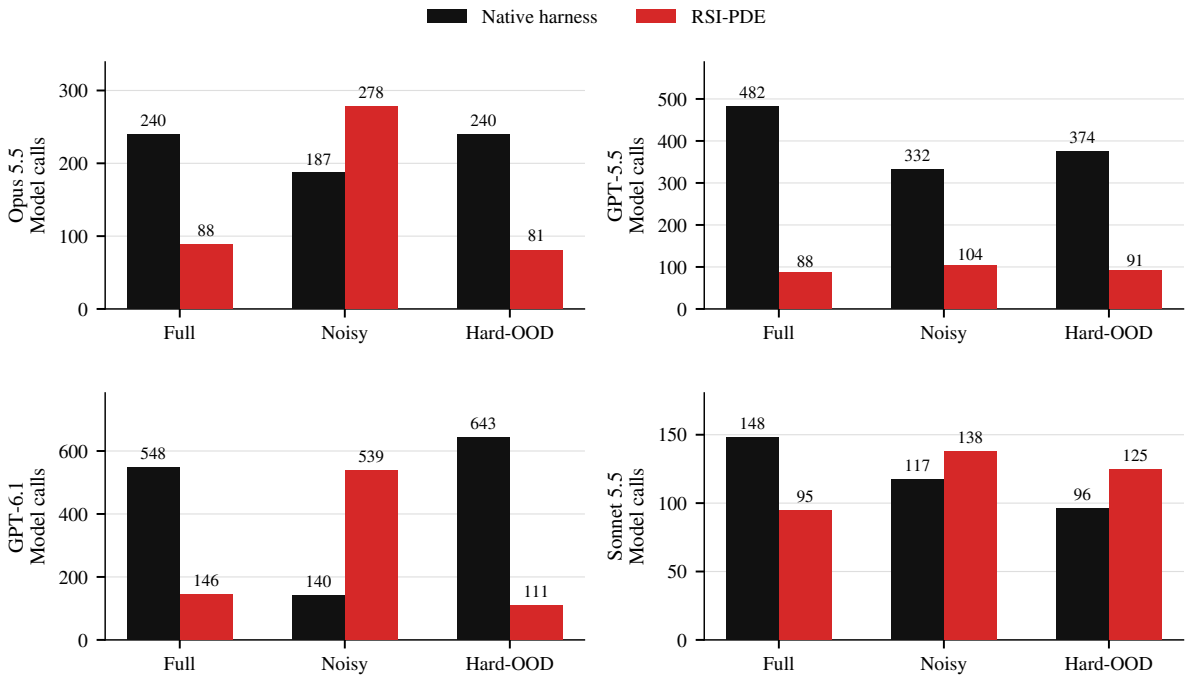

Figure 6 | Recorded model calls by model and setting. Black: native harness; red: RSI-PDE. Calls are summed across development groups and are not replaced by stop-rule replay counts. The corresponding dollar figures have a different provenance, as explained in the text and Table 6.

5.5. Residual diagnostics and their interpretation

Residual-guided identification in Section 4.2 uses a hypothesized equation. The residual table on the website instead evaluates the frozen returned solution under the evaluator's *true* equation. If its terms are $T_j[\hat{u}]$, the reported relative residual is

$$r_{\text{true}}(\hat{u}) = \frac{\| \sum_j T_j[\hat{u}] \|}{\sum_j \|T_j[\hat{u}]\|}. \tag{6}$$

This diagnostic needs no reference solution, but does need the true class and coefficients; it is computed afterward and is never exposed as a hidden-equation selection oracle to the agent.

The calculation uses spectral derivatives in periodic directions, fourth-order differences on bounded grids, cubic splines for time derivatives and an L1 approximation for Caputo derivatives. Darcy uses a weak residual against sine modes. The point-cloud elliptic family has no reported residual. These discretizations create a resolution-dependent floor, so a residual measures equation consistency at the returned resolution, not a uniform solution-error bound.

For example, the historical Full Opus 5.5 native and RSI rows have median relative L^2 errors of roughly 4.5×10^{-4} and 4.3×10^{-4}, but median residuals of 3.9×10^{-4} and 4.6×10^{-4}. A lower aggregate residual therefore need not correspond to a lower aggregate reference error. The reported rank correlations and passed/failed medians provide additional information, but their coverage includes only the cases for which the corresponding diagnostics are available.

The Full native GPT-6.1 residual row retains 199/210 reference-run passes, and Sonnet 5.5 retains 193/210, rather than the updated headline totals 189 and 183. Single-run residual rows can also have different sample coverage from the headline averages. We label these as reference-run diagnostics in Section F, preserving the source values instead of presenting them as new measurements of the updated totals.

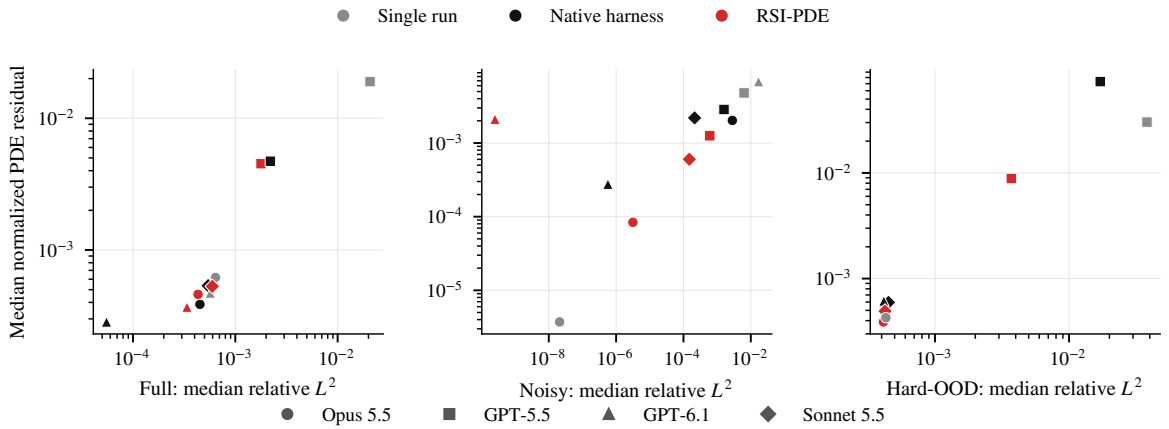

Figure 7 | Each point pairs a website row's median relative L^2 error with its median post-hoc true-equation residual. Points are aggregate medians, not paired per-case observations; marker shapes identify models and colors identify harnesses. Coverage and reference runs vary, including historical Full native diagnostics. The plot illustrates complementary diagnostics and should not be read as a causal residual-to-accuracy law.

6. Discussion and Limitations

Unknown physics as a controlled scientific task. Jointly hiding equation family and coefficients makes solver construction harder than translating a fully specified PDE into code. Full, Noisy and Hard-OOD expose different failure modes in that process. Nevertheless, the task supplies a finite candidate library and known physical inputs. The 1,001-case collection is mostly synthetic, covers one- and two-dimensional domains, and includes selected data-driven input fields. It is a reproducible analogue of scientific modeling, not a complete representation of noisy laboratory data, open-ended equation discovery or industrial workloads. Noisy selection and Hard-OOD feasibility filtering further restrict the evaluated populations.

Residual guidance and identifiability. A wrong equation can have a small residual when observations are sparse, weakly informative or differentiated inaccurately. Multiple families or parameter values may fit the same samples, especially at 50% noise. Keeping five candidates reduces premature commitment but does not guarantee true-family retention. The reported experiments contain no isolated residual-screening ablation, shortlist-recall measurement or parameter-recovery rate. Consequently, improved solution pass rates support the complete workflow, without establishing that residual guidance alone is faster or recovers every true coefficient. Reference-based L^2 checks also inherit interpolation and numerical-reference error; self-convergence does not prove a uniform reference error bound.

Replication and benchmark reuse. Most model–setting comparisons have one run; the three-run Full Opus native range is large relative to several margins. Repeated seeds and paired initial states are needed for uncertainty estimates and component ablations. Running agents cannot see hidden test feedback, but maintainers have inspected benchmark results when selecting design choices, including rules for using known physical functions. The current record is therefore developmental rather than an untouched holdout estimate (Dwork et al., 2015; Blum & Hardt, 2015). Noise-level visibility also differs among the reported Noisy runs: GPT-6.1 and Sonnet 5.5 hide it, while other displayed Noisy rows show it; Hard-OOD hides it throughout. Controlled comparisons should match those conditions and use fresh committed test draws. Historical Full diagnostics are labeled separately from updated headline totals.

What evolves and what the resource results mean. The model weights and executable supervisor remain fixed, while solver code, policy text and memory can change. Final-program improvement does not establish that an evolved controller is a better improver; that claim requires matched continuation experiments. Recorded calls decrease in eight of 12 comparisons, with substantial exceptions on Noisy. Dollar costs depend on token usage and caching, and selected costs use retrospective stop-rule replay while retaining full-budget scores. Those estimates require fresh stopped runs before they can support jointly achieved accuracy/cost claims.

7. Conclusion

RSI-PDEbench brings together 1,001 PDE/FPDE cases and an unknown-family, unknown-parameter identify-then-solve task. Full, Noisy and Hard-OOD separately test noise-free accuracy, robustness to severe Gaussian sample noise, and transfer to unseen templates and shifted coefficients. The RSI-PDE harness uses residuals to prioritize candidate physical laws, then combines public relative L^2 feedback, coefficient refinement and validation to retain accurate executable solvers and improve their policy and memory. The reported four-model comparison improves mean pass rates over single-run and native-harness baselines in all three settings and reduces aggregate recorded model calls. Repeated runs, residual-specific ablations and fresh hidden evaluations will clarify which parts of this workflow generalize most reliably.

Data and Code Availability

The RSI-PDEbench dataset and RSI-PDE implementation are publicly available at https://github.com/jeremylu916/RSI-PDE. The release includes 1,001 PDE and FPDE cases spanning 42 templates, together with case-generation routines, reference solutions, evaluation code, and the RSI-PDE harness. The evaluation settings and split-construction procedures are described in Section 3. The frozen result snapshot and figure-generation scripts support reproduction of the reported tables and plots, as detailed in Appendix D. Benchmark results and additional documentation are available at https://jeremylu916.github.io/RSI-PDE/.

Acknowledgment

We gratefully acknowledge the support from DARPA MATHBAC program under grant number: HR0011-26-3-E152, the National Science Foundation (DMS-2533878, DMS-2053746, DMS-2134209, ECCS-2328241, CBET-2347401 and OAC-2311848), and U.S. Department of Energy (DOE) Office of Science Advanced Scientific Computing Research program under the "Uncertainty Quantification for Multifidelity Operator Learning (MOLUcQ)" project (Project No. 81739), DE-SC0023161, the SciDAC LEADS Institute, and DOE–Fusion Energy Science, under grant number: DE-SC0024583.

References

Bai, Y., Lu, B., Liu, Y., Akbari, E., Kolouri, S., Wang, L., He, P., Wang, S., Zhang, R., and Lin, G. SE(3)-MeanFlow: Few-step protein backbone generation on Lie groups. *arXiv preprint arXiv:2607.27431*, 2026.

Blum, A. and Hardt, M. The ladder: A reliable leaderboard for machine learning competitions. In *Proceedings of the 32nd International Conference on Machine Learning*, pp. 1006–1014, 2015.

Brunton, S. L., Proctor, J. L., and Kutz, J. N. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016.

Cao, L., Lu, B., Shen, Y., and Guo, Y. MedRouter: Demystifying knowledge differences across medical LLMs for routing-based reasoning. *arXiv preprint arXiv:2609.33119*, 2026.

Chan, J. S., Chowdhury, N., Jaffe, O., Aung, J., Sherburn, D., Mays, E., Starace, G., Liu, K., Maksin, L., Patwardhan, T., Weng, L., and Madry, A. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025.

Deng, Z., Lu, B., Feng, H., Huang, S., Wang, D., Xu, Y., Zhang, Z., Sun, Y., Mou, C., Zhang, R., et al. CONTRAMEM: Learning self-evolving procedural memory from contrasting multi-model trajectories. *arXiv preprint arXiv:2608.22533*, 2026.

Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., and Roth, A. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015.

Gupta, J. K. and Brandstetter, J. Towards multi-spatiotemporal-scale generalized PDE modeling. *arXiv preprint arXiv:2209.15616*, 2022.

Hang, Z., Yashengjiang, Y., Li, J., Dong, H., Wei, Y., Hao, Z., Ma, J., Bai, S., Kai, H., Yue, X., Si, G., Jiang, D., Yao, C., Hu, Z., Zhang, J., Liu, P., Shen, Y., Ren, X., Liu, L., Xu, Z., Li, H., Yao, Q., Dong, H., and Wang, H. PDEAgent-Bench: A multi-metric, multi-library benchmark for PDE solver generation. *arXiv preprint arXiv:2605.09636*, 2026. URL https://arxiv.org/abs/2605.09636.

Hao, Z., Yao, J., Su, C., Su, H., Wang, Z., Lu, F., Xia, Z., Zhang, Y., Liu, S., Lu, L., and Zhu, J. PINNacle: A comprehensive benchmark of physics-informed neural networks for solving PDEs. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.

Hu, S., Lu, C., and Clune, J. Automated design of agentic systems. In *International Conference on Learning Representations*, 2025.

Huang, J., Yang, G., Wang, Z., and Park, J. J. DiffusionPDE: Generative PDE-solving under partial observation. In *Advances in Neural Information Processing Systems*, 2024.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

Li, S., Marwah, T., Shen, J., Sun, W., Risteski, A., Yang, Y., and Talwalkar, A. CodePDE: An inference framework for LLM-driven PDE solver generation. *Transactions on Machine Learning Research*, 2026a.

Li, X., Hou, J., Deng, Z., Zhang, Z., Li, T., Lu, B., Hu, B., Zhao, Y., and Hao, Y. Chain of Risk: Safety failures in large reasoning models and mitigation via adaptive multi-principle steering. arXiv preprint arXiv:2605.05678, 2026b. URL https://arxiv.org/abs/2605.05678.

Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

Long, Z., Lu, Y., Ma, X., and Dong, B. PDE-Net: Learning PDEs from data. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 3208–3216, 2018.

Lu, B., Moya, C., and Lin, G. NSGA-PINN: A multi-objective optimization method for physics-informed neural network training. *Algorithms*, 16(4):194, 2023.

Lu, B., Hao, Z., Moya, C., and Lin, G. FPINN-DeepONet: A physics-informed operator learning framework for multi-term time-fractional mixed diffusion-wave equations. *Journal of Computational Physics*, 538:114184, 2025.

Lu, B., Deng, Z., Zhang, R., Hu, B., Zhao, Y., Tian, Y., Mou, C., Lin, G., and Li, X. Muon-OGD: Muon-based spectral orthogonal gradient projection for LLM continual learning. *arXiv preprint arXiv:2605.08949*, 2026a.

Lu, B., Mou, C., and Lin, G. iPINNER: An iterative physics-informed neural network with ensemble Kalman filter. *Journal of Computational Physics*, 548:114592, 2026b. doi: 10.1016/j.jcp.2025.114592.

Lu, B., Mou, C., and Lin, G. Morephy-Net: An evolutionary multi-objective optimization for replica-exchange-based physics-informed neural operator learning networks. *Computer Methods in Applied Mechanics and Engineering*, 460:119123, 2026c.

Lu, B., Zhang, J., and Lin, G. Muon with spectral guidance: Efficient optimization for scientific machine learning. *Journal of Computational Physics*, 565:115231, 2026d. doi: 10.1016/j.jcp.2026.115231.

Lu, B., Zhang, R., Mou, C., Li, N., and Lin, G. AdamFLIP: Adaptive momentum feedback linearization optimization for hard constrained PINN training. *arXiv preprint arXiv:2605.08408*, 2026e.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.

Mou, C., Lu, B., and Lin, G. Neural-POD: A plug-and-play neural operator framework for infinite-dimensional functional nonlinear proper orthogonal decomposition. *arXiv preprint arXiv:2602.15632*, 2026.

Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.-S., et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

Ohana, R., McCabe, M., Meyer, L., Morel, R., Agocs, F. J., et al. The well: a large-scale collection of diverse physics simulations for machine learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.

Raissi, M., Perdikaris, P., and Karniadakis, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J. R., Ellenberg, J. S., Wang, P., Fawzi, O., Kohli, P., and Fawzi, A. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

Rudy, S. H., Brunton, S. L., Proctor, J. L., and Kutz, J. N. Data-driven discovery of partial differential equations. *Science Advances*, 3(4):e1602614, 2017.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Shojaee, P., Meidani, K., Gupta, S., Farimani, A. B., and Reddy, C. K. LLM-SR: Scientific equation discovery via programming with large language models. In *International Conference on Learning Representations*, 2025.

Takamoto, M., Praditia, T., Leiteritz, R., MacKinlay, D., Alesiani, F., Pflüger, D., and Niepert, M. PDEBench: An extensive benchmark for scientific machine learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 35, pp. 1596–1611, 2022.

Tian, M., Gao, L., Zhang, S. D., Chen, X., Fan, C., Guo, X., et al. SciCode: A research coding benchmark curated by scientists. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.

Tian, Y., Hu, B., Wang, H., Lu, B., and Wu, F. Harness evolution hits a ceiling: When weight training should begin. *arXiv preprint arXiv:2610.11655*, 2026a.

Tian, Y., Hu, B., Wu, F., Li, X., Lu, B., and Gong, N. Z. When think-with-image meets safety: What determines multimodal jailbreak robustness? *arXiv preprint arXiv:2605.27932*, 2026b.

Wijk, H., Lin, T., Becker, J., Jawhar, S., Parikh, N., Broadley, T., Chan, L., Chen, M., Clymer, J., Dhyani, J., et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 2025.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

Zelikman, E., Lorch, E., Mackey, L., and Kalai, A. T. Self-taught optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling*, 2024.

A. Related Work

PDE learning and equation discovery. PDEBench, PDEArena, PINNacle and The Well provide important test beds for learned numerical models (Takamoto et al., 2022; Gupta & Brandstetter, 2022; Hao et al., 2024; Ohana et al., 2024). Physics-informed neural networks use equation residuals in forward and inverse problems (Raissi et al., 2019), while neural operators learn solution mappings across problem instances (Li et al., 2021). Equation discovery instead seeks a governing law: sparse identification and PDE-FIND select differential terms from data (Brunton et al., 2016; Rudy et al., 2017), and PDE-Net learns differential operators (Long et al., 2018). RSI-PDEbench combines identification with construction of an executable numerical solver. The entrant can use classical, learned or hybrid methods, but is graded on the returned solution. Five data-driven templates reuse input fields from DiffusionPDE (Huang et al., 2024), with reference solutions recomputed for this benchmark.

Physics-informed numerical learning. Training methods address complementary challenges in constructing accurate scientific surrogates. NSGA-PINN uses multi-objective optimization for physics-informed network training (Lu et al., 2023), iPINNER combines iterative physics-informed learning with ensemble Kalman filtering (Lu et al., 2026b), and Morephy-Net studies evolutionary multi-objective, replica-exchange physics-informed operator learning (Lu et al., 2026c). FPINN-DeepONet targets multi-term time-fractional mixed diffusion–wave equations (Lu et al., 2025), while Neural-POD develops a neural-operator formulation of nonlinear proper orthogonal decomposition (Mou et al., 2026). Spectrally guided Muon optimization and AdamFLIP study optimization for scientific learning and hard-constrained PINNs (Lu et al., 2026d;e). These works provide numerical-learning approaches relevant to the solution methods an agent may investigate. RSI-PDEbench evaluates the broader process of identifying the equation, estimating its coefficients and constructing a solver under a fixed development budget.

Scientific coding and agent interfaces. Scientific and software-engineering benchmarks evaluate programs, research tasks or repository-level changes (Tian et al., 2024; Jimenez et al., 2024; Chan et al., 2025; Wijk et al., 2025). SWE-agent demonstrates the importance of the interface through which a model edits and executes code (Yang et al., 2024). CodePDE studies inference-time generation and refinement of solvers for specified equations (Li et al., 2026a). PDEAgent-Bench emphasizes accurate, efficient solver generation for known PDE specifications and compatibility with multiple FEM libraries (Hang et al., 2026). RSI-PDEbench complements that emphasis by hiding the instance's family and coefficients and including fractional dynamics. LLM-SR searches for scientific equations represented as programs (Shojaee et al., 2025). Our comparison combines finite-library equation identification with solver development, keeps the foundation model fixed within a model-specific comparison, and varies the harness under numerical feedback and a recorded development budget. Unlike a unit-test-only task, a solution must also meet a quantitative error tolerance over an evaluation grid.

Iterative and self-improving agents. Reflexion and Self-Refine use feedback to revise behavior or outputs without retraining model weights (Shinn et al., 2023; Madaan et al., 2023). Program-search systems use evaluations to improve candidate code (Romera-Paredes et al., 2024; Novikov et al., 2025), and other systems optimize improvement procedures or agent designs (Zelikman et al., 2024; Hu et al., 2025). CONTRAMEM studies self-evolving procedural memory from contrasting multi-model trajectories (Deng et al., 2026), and work on the limits of harness evolution examines when weight training becomes necessary (Tian et al., 2026a). Muon-OGD studies spectral orthogonal gradient projection for LLM continual learning (Lu et al., 2026a). In our setting, the foundation weights remain fixed, and RSI-PDE updates solver code, an instruction policy and memory. The displayed benchmark runs keep the executable supervisor fixed. Establishing that an evolved controller is a better improver would require paired continuations from the same solver and memory, beyond a comparison of final programs. Adaptive reuse of validation or test results also requires care (Dwork et al., 2015; Blum & Hardt, 2015); we therefore state the provenance of the development results explicitly.

Structured scientific applications and agent reliability. Broader scientific-learning applications include SE(3)-MeanFlow for protein-backbone generation on Lie groups (Bai et al., 2026) and MedRouter for routing-based reasoning across medical language models (Cao et al., 2026). They illustrate structured generation and task-specific routing in domains beyond PDE solver construction. Agent reliability also has dimensions beyond numerical accuracy: Chain of Risk studies safety failures and adaptive steering in reasoning models (Li et al., 2026b), while Think-with-Image studies multimodal jailbreak robustness (Tian et al., 2026b). Those questions complement the execution isolation, information boundaries and quantitative solution checks used here.

B. Templates

Table 5 lists the 42 templates. In Full, every template has one training case, one validation case and five hidden-test cases; the other settings have the allocations in Table 2. Templates marked † were hardened after calibration (Section C); templates marked ‡ take their input fields from the DiffusionPDE test set.

Table 5 | The 42 templates. Dim.: spatial dimension (0 for the fractional ODE). Steady: no time dependence.

ID	Template	Class	Dim.	Steady
Classical and fractional templates				
1a	Poisson, manufactured	Poisson	1	yes
1b	Poisson, multiscale	Poisson	1	yes
1c	Poisson, interior layer	Poisson	1	yes
2a	Heat, manufactured	heat	1	
2b	Heat, free decay	heat	1	
2c	Heat, periodic source	heat	1	
3a	Conv.–diff., manufactured	convection–diffusion	1	
3b†	Conv.–diff., boundary layer	convection–diffusion	1	yes
3c†	Conv.–diff., layer + source	convection–diffusion	1	yes
3d	Conv.–diff., Gaussian pulse	convection–diffusion	1	
4a	Burgers, manufactured	Burgers	1	
4b	Burgers, sine data	Burgers	1	
4c	Burgers, viscous shock	Burgers	1	
5	Wave 2-D	wave	2	
6†	Fractional ODE	Caputo ODE	0	
7	Time-fractional diffusion	Caputo	1	
8	Multi-term fractional	multi-term Caputo	1	
9†	Fractional, zero initial data	Caputo	1	
10	Variable order $\alpha(t)$	variable order	1	
11†	Variable order $\alpha(x,t)$	variable order	1	
Nonlinear and multiphysics templates				
12a	Kuramoto–Sivashinsky, manufactured	Kuramoto–Sivashinsky	1	
12b	Kuramoto–Sivashinsky, chaotic	Kuramoto–Sivashinsky	1	
13a	Navier–Stokes, Taylor–Green	Navier–Stokes	2	
13b	Navier–Stokes, Kovasznay	Navier–Stokes	2	yes
14a	Cahn–Hilliard, manufactured	Cahn–Hilliard	2	
14b	Cahn–Hilliard, spinodal	Cahn–Hilliard	2	
15a	Maxwell, cavity mode	Maxwell TE	2	
15b	Maxwell, current-driven	Maxwell TE	2	
16a	Time-harmonic EM, low frequency	curl–curl	2	yes
16b	Time-harmonic EM, high frequency	curl–curl	2	yes
Data-driven and additional-dimensional templates				
17	Burgers, Gaussian data†	Burgers	1	
18	Burgers, random data‡	Burgers	1	
19	Conv.–diff., periodic pulse	convection–diffusion	1	
20	Darcy flow‡	Darcy	2	yes
21	Poisson, random forcing‡	Poisson 2-D	2	yes
22	Helmholtz†‡	Helmholtz	2	yes
23	Navier–Stokes vorticity‡	vorticity Navier–Stokes	2	
24	Kolmogorov flow	vorticity Navier–Stokes	2	
25	Allen–Cahn	Allen–Cahn	2	
26	Burgers 2-D	Burgers 2-D	2	
27	Advection–diffusion–reaction	ADR	2	
28	Elliptic equation	elliptic	2	yes

C. Reference Construction and Calibration

Analytic references are evaluated directly at the evaluator's nodes. Numerical references are constructed with a finer computation than the comparison resolution and checked after both are mapped to the common evaluation grid. Representative settings in the reference builders are:

- chaotic Kuramoto–Sivashinsky: 512 Fourier nodes and 120 ETDRK4 substeps per output interval, compared with 256 nodes and 60 substeps;
- spinodal Cahn–Hilliard: 256 nodes per spatial direction and 100 substeps, compared with 128 nodes and 50 substeps;
- Gaussian-data Burgers: refined finite-volume computations at 2048 and 1024 spatial nodes;
- Poisson and Helmholtz input-field problems: fine/coarse spatial settings of 1020 and 510;
- the Elliptic equation reference: a finite-volume discretization with a preconditioned conjugate-gradient relative tolerance of 10^{-11}.

The reference scheme and error allowance are fixed by the benchmark, not changed to favor a submitted solver. Higher order, finer resolution or tighter linear-solver tolerance are used where appropriate to the family; these settings are not a claim that every reference has the same discretization error.

The public website also includes calibration profiles from fixed probe programs. Its displayed probe profiles cover 640 variants of 32 templates; the additional multiphysics templates complete the 42-template benchmark. Calibration informs hardening and the diagnostic Noisy selection. Since subset selection changes the population being evaluated, the Full, Noisy and Hard-OOD pass rates should not be interpreted as estimates of the same underlying case distribution.

D. Implementation and Reproduction

Recorded source snapshot. The numerical tables and plots are generated from `arxiv/data/website_results_20261107.json`, a frozen copy of the public website after revision `76b33da`. The generator is `arxiv/make_website_assets.py`. It does not run an agent or inspect hidden per-case outcomes. The website corrections are presentation-level provenance events; the original experiment records are retained. The authors clarified that Full denotes the noise-free `full0` sampling variant, which sets observation noise to zero. The frozen website snapshot retains its published `full` presentation keys and reported totals. This protocol clarification is recorded in `arxiv/data/protocol_corrections.json`; it does not overwrite original experiment records.

Agent-level residuals and supervisor. Candidate-equation residuals are implemented by model-written sandbox routines. For example, a retained fractional solver uses least-squares terms for PDE collocation, sample fit and boundaries before comparing candidate scores; another solver checks coefficient consistency against known forcing and initial data. These routines are distinct from evaluator-only true-equation residuals. The top-five policy describes the identification workflow; the aggregate results do not isolate its effect.

The supervisor is implemented in `src/pde_rsi/code_agents.py`; the standard gate is in `src/pde_rsi/evaluator.py`, and the native wrapper in `scripts/run_native_agent.py`. The public-pass rule compares the same public case identifiers before and after an update, gains at least one pass, loses none, permits no new execution/constraint/resource failure and bounds error changes on cases that already pass. Model weights and executable control flow stay fixed during the reported runs. Raw configuration identifiers remain in the result snapshot for reproduction, without defining separate methods in this paper.

Reproducible paper build. The revised arXiv manuscript is maintained directly in this directory, retaining the original document style and author block. Asset generation uses the frozen website JSON, and the LaTeX build uses the included style, bibliography and figures.

E. Additional Learning Curves and Resource Records

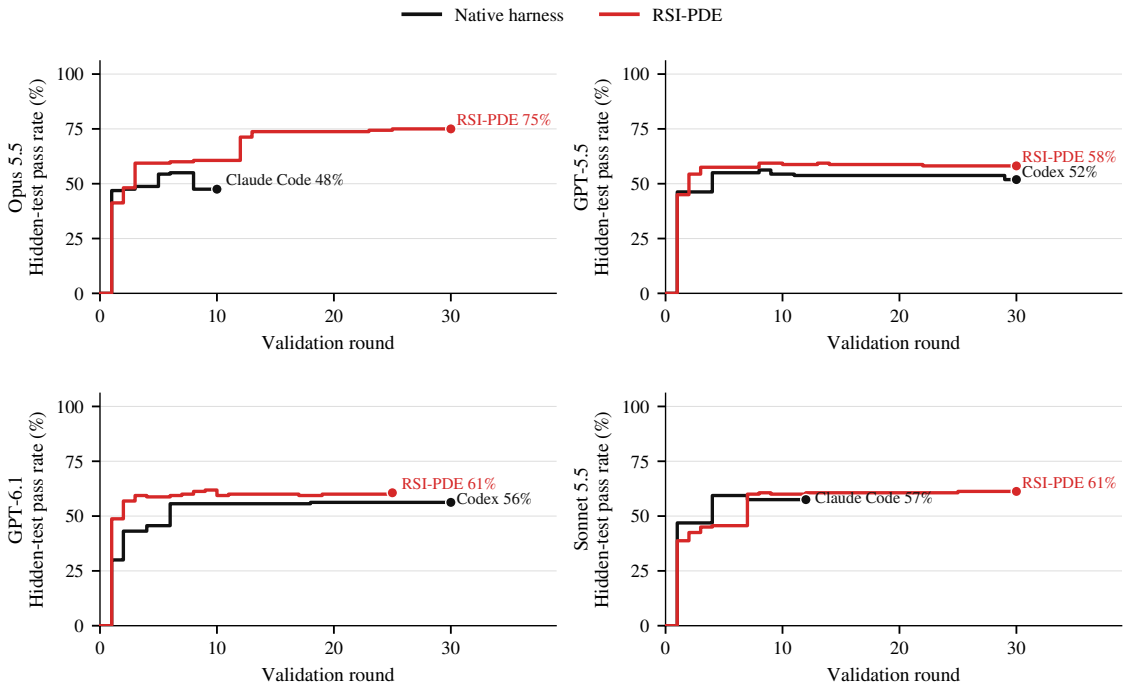

Figure 8 | Noisy trajectories for all four models, with corrected GPT-6.1 identities: red RSI-PDE finishes at 97/160 and black Codex at 90/160. The curves retain the supplied numerical measurements and change only their corrected harness attribution. Round indices are validation-submission indices.

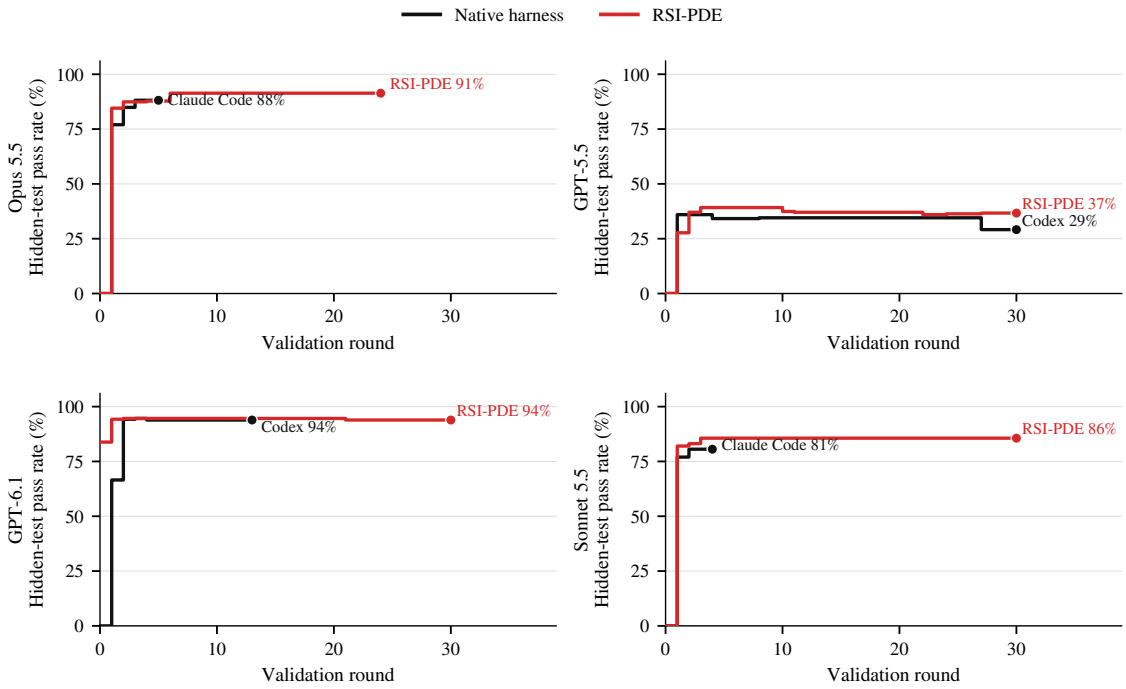

Figure 9 | Hard-OOD trajectories. RSI-PDE is red and the native harness is black. Template novelty and coefficient shift are both included in the 278-case aggregate; Table 4 separates their endpoints.

Table 6 | Website resource records, summed over the development groups and averaged where repeated. Recorded rounds are summed group submissions, not the final horizontal coordinate of a combined curve. †: updated Full native total paired with historical resource records. ‡: corrected Noisy GPT-6.1 attribution. ∗: retrospective stop-rule replay cost; calls and accuracy remain full-budget values.

Setting	Model	Entrant	Passed	Calls	Recorded rounds	Cost ($)
Full	Opus 5.5	Single run	66.3	3	0	7.32
	Opus 5.5	Claude Code	193	240	20	16.74
	Opus 5.5	RSI-PDE	196	88	46	30.47∗
	GPT-5.5	Single run	38.3	3	0	1.22
	GPT-5.5	Codex	95	482	87	40.97
	GPT-5.5	RSI-PDE	104	88	64	32.01
	GPT-6.1	Single run	77.8	4	0	–
	GPT-6.1	Codex†	189	548	20	–
	GPT-6.1	RSI-PDE	193	146	67	–
	Sonnet 5.5	Single run	59	3	0	–
	Sonnet 5.5	Claude Code†	183	148	10	6.41
	Sonnet 5.5	RSI-PDE	187	95	50	21.81∗
Noisy	Opus 5.5	Single run	37	3	0	3.35
	Opus 5.5	Claude Code	76	187	24	13.90
	Opus 5.5	RSI-PDE	120	278	68	67.10∗
	GPT-5.5	Single run	51.5	3	0	1.15
	GPT-5.5	Codex	83	332	78	20.58
	GPT-5.5	RSI-PDE	93	104	81	17.84∗
	GPT-6.1	Single run	45.3	3	0	–
	GPT-6.1	Codex‡	90	140	82	–
	GPT-6.1	RSI-PDE‡	97	539	47	–
	Sonnet 5.5	Single run	36	3	0	–
	Sonnet 5.5	Claude Code	92	117	21	4.21
	Sonnet 5.5	RSI-PDE	98	138	90	25.29∗
Hard-OOD	Opus 5.5	Single run	133	3	0	7.61
	Opus 5.5	Claude Code	245	240	14	18.86
	Opus 5.5	RSI-PDE	254	81	48	17.55∗
	GPT-5.5	Single run	42	3	0	1.06
	GPT-5.5	Codex	81	374	89	22.79
	GPT-5.5	RSI-PDE	102	91	64	22.88
	GPT-6.1	Single run	135	3	0	–
	GPT-6.1	Codex	261	643	35	–
	GPT-6.1	RSI-PDE	261	111	79	–
	Sonnet 5.5	Single run	104	3	0	–
	Sonnet 5.5	Claude Code	224	96	10	3.33
	Sonnet 5.5	RSI-PDE	238	125	85	10.55∗

F. Retrospective Residual Tables

The following tables reproduce the website's aggregate residual diagnostics. They use the reference-run pass counts belonging to those diagnostic records. Medians cover cases that returned numerical solution arrays, including solutions that failed the accuracy test. Residual coverage can be smaller, and point-cloud elliptic outputs have no residual. The last column is the reported Spearman correlation between available per-case relative L^2 errors and residuals, rather than a correlation computed from the plotted row medians.

Table 7 | Full reference-run diagnostics. †: historical native counts 199/210 for GPT-6.1 and 193/210 for Sonnet 5.5; the current headline totals are 189 and 183. Single-run coverage and sample means can also differ from the headline comparison. These rows are not new measurements of the updated totals.

Model	Entrant	Ref. passes/cases	Median L^2	Residual	Residual, passed	Residual, failed	ρ
Opus 5.5	Single run	68.3/100	6.4×10^{-4}	6.2×10^{-4}	6.2×10^{-4}	7.1×10^{-4}	0.70
Opus 5.5	Claude Code	193/210	4.5×10^{-4}	3.9×10^{-4}	3.4×10^{-4}	1.9×10^{-3}	0.71
Opus 5.5	RSI-PDE	196/210	4.3×10^{-4}	4.6×10^{-4}	4.3×10^{-4}	1.5×10^{-3}	0.71
GPT-5.5	Single run	39.8/190	2.1×10^{-2}	1.9×10^{-2}	7.9×10^{-4}	4.3×10^{-2}	0.87
GPT-5.5	Codex	95/210	2.2×10^{-3}	4.7×10^{-3}	1.2×10^{-3}	1.7×10^{-2}	0.78
GPT-5.5	RSI-PDE	104/210	1.8×10^{-3}	4.5×10^{-3}	7.4×10^{-4}	1.5×10^{-2}	0.85
GPT-6.1	Single run	103.7/116.7	5.7×10^{-4}	4.7×10^{-4}	3.8×10^{-4}	1.3×10^{-3}	0.77
GPT-6.1	Codex	199/210†	5.5×10^{-5}	2.8×10^{-4}	2.2×10^{-4}	1.9×10^{-3}	0.68
GPT-6.1	RSI-PDE	193/210	3.4×10^{-4}	3.7×10^{-4}	2.7×10^{-4}	2.1×10^{-3}	0.77
Sonnet 5.5	Claude Code	193/210†	5.4×10^{-4}	5.4×10^{-4}	5.2×10^{-4}	1.5×10^{-3}	0.66
Sonnet 5.5	RSI-PDE	187/210	6.0×10^{-4}	5.3×10^{-4}	4.1×10^{-4}	2.0×10^{-3}	0.73

Table 8 | Noisy reference-run diagnostics. The GPT-6.1 harness identities follow the author's correction; residual values and their original numerical coverage are preserved.

Model	Entrant	Ref. passes/cases	Median L^2	Residual	Residual, passed	Residual, failed	ρ
Opus 5.5	Single run	37/42	2.1×10^{-8}	3.7×10^{-6}	2.3×10^{-6}	1.3×10^{-3}	0.86
Opus 5.5	Claude Code	76/160	2.8×10^{-3}	2.0×10^{-3}	6.9×10^{-5}	1.3×10^{-2}	0.90
Opus 5.5	RSI-PDE	120/160	3.2×10^{-6}	8.4×10^{-5}	5.1×10^{-5}	1.2×10^{-2}	0.89
GPT-5.5	Single run	51.5/160	6.3×10^{-3}	4.8×10^{-3}	1.6×10^{-3}	1.5×10^{-2}	0.76
GPT-5.5	Codex	83/160	1.6×10^{-3}	2.9×10^{-3}	5.5×10^{-4}	1.5×10^{-2}	0.92
GPT-5.5	RSI-PDE	93/160	6.0×10^{-4}	1.3×10^{-3}	2.3×10^{-4}	1.1×10^{-2}	0.84
GPT-6.1	Single run	45.3/124.3	1.7×10^{-2}	6.8×10^{-3}	8.7×10^{-5}	2.1×10^{-2}	0.94
GPT-6.1	Codex	90/160	5.7×10^{-7}	2.8×10^{-4}	5.6×10^{-5}	3.1×10^{-2}	0.93
GPT-6.1	RSI-PDE	97/160	2.5×10^{-10}	2.1×10^{-3}	4.3×10^{-4}	9.4×10^{-3}	0.79
Sonnet 5.5	Claude Code	92/160	2.1×10^{-4}	2.2×10^{-3}	6.0×10^{-5}	1.1×10^{-2}	0.91
Sonnet 5.5	RSI-PDE	98/160	1.5×10^{-4}	6.0×10^{-4}	5.4×10^{-5}	1.3×10^{-2}	0.96

Table 9 | Hard-OOD reference-run diagnostics, including the available single-run rows. The true-equation residual is an evaluator-only post-hoc measurement and does not alter the relative-L^2 pass criterion.

Model	Entrant	Ref. passes/cases	Median L^2	Residual	Residual, passed	Residual, failed	ρ
Opus 5.5	Single run	133/133	4.3×10^{-4}	4.3×10^{-4}	4.3×10^{-4}	–	0.60
Opus 5.5	Claude Code	245/278	4.2×10^{-4}	4.1×10^{-4}	3.4×10^{-4}	2.4×10^{-2}	0.66
Opus 5.5	RSI-PDE	254/278	4.1×10^{-4}	3.9×10^{-4}	3.6×10^{-4}	3.6×10^{-2}	0.62
GPT-5.5	Single run	42/278	3.8×10^{-2}	3.0×10^{-2}	7.7×10^{-4}	5.3×10^{-2}	0.82
GPT-5.5	Codex	81/278	1.7×10^{-2}	7.3×10^{-2}	4.6×10^{-4}	1.5×10^{-1}	0.87
GPT-5.5	RSI-PDE	102/278	3.7×10^{-3}	8.8×10^{-3}	6.0×10^{-4}	1.0×10^{-1}	0.90
GPT-6.1	Codex	261/278	4.2×10^{-4}	6.1×10^{-4}	5.7×10^{-4}	3.8×10^{-3}	0.38
GPT-6.1	RSI-PDE	261/278	4.2×10^{-4}	4.6×10^{-4}	4.2×10^{-4}	1.4×10^{-2}	0.62
Sonnet 5.5	Claude Code	224/278	4.5×10^{-4}	6.0×10^{-4}	4.5×10^{-4}	2.2×10^{-2}	0.65
Sonnet 5.5	RSI-PDE	238/278	4.2×10^{-4}	4.9×10^{-4}	4.6×10^{-4}	5.8×10^{-2}	0.59